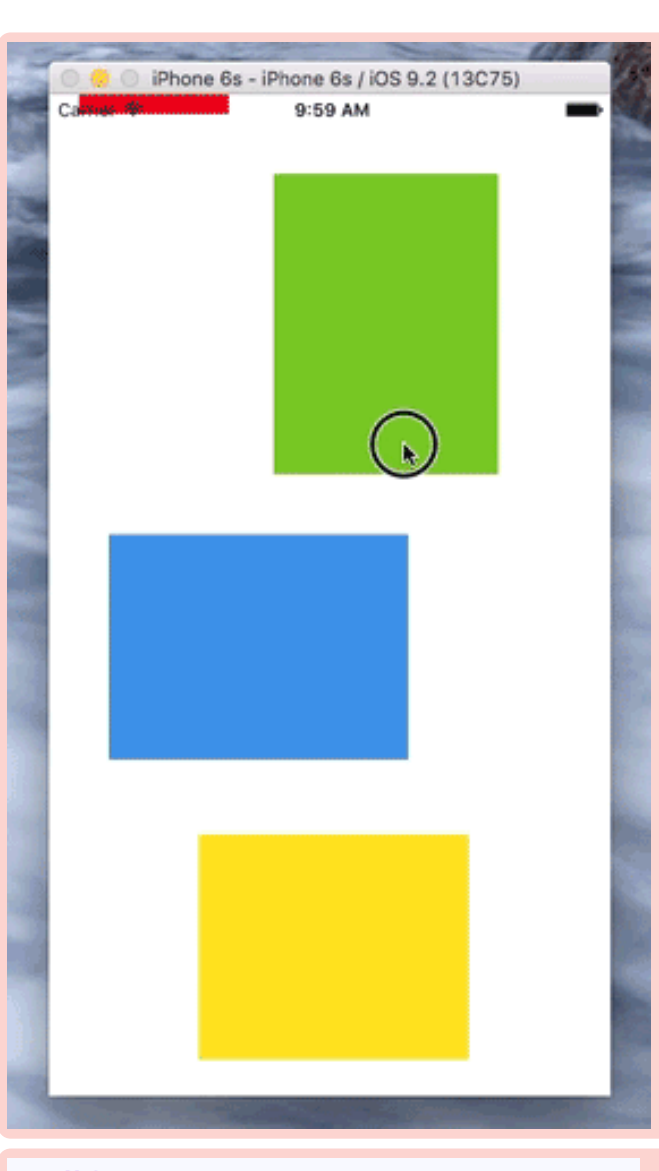
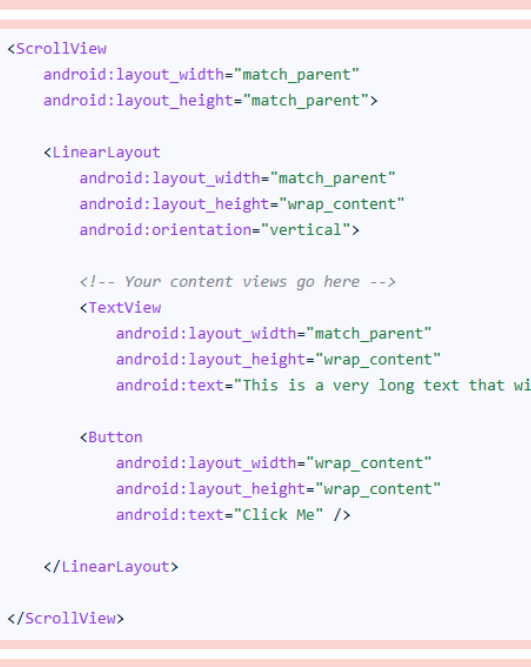
Lab 4 Scrollable View

① Scroll View

Look

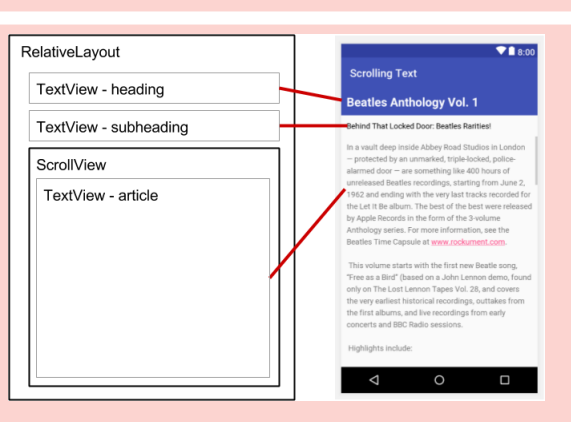


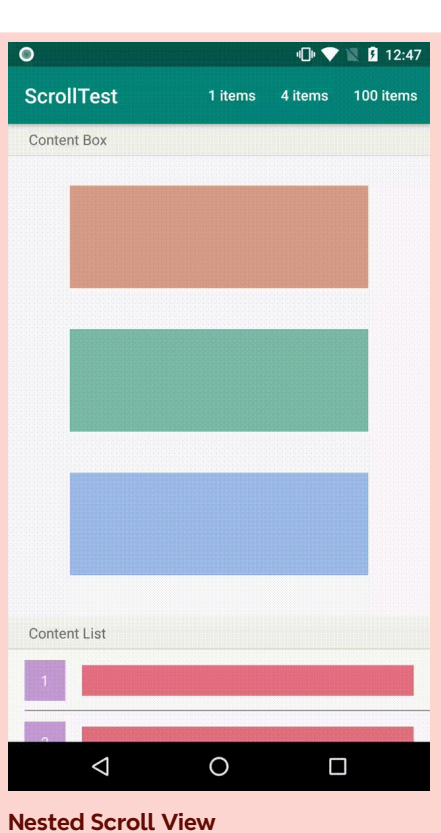
Code



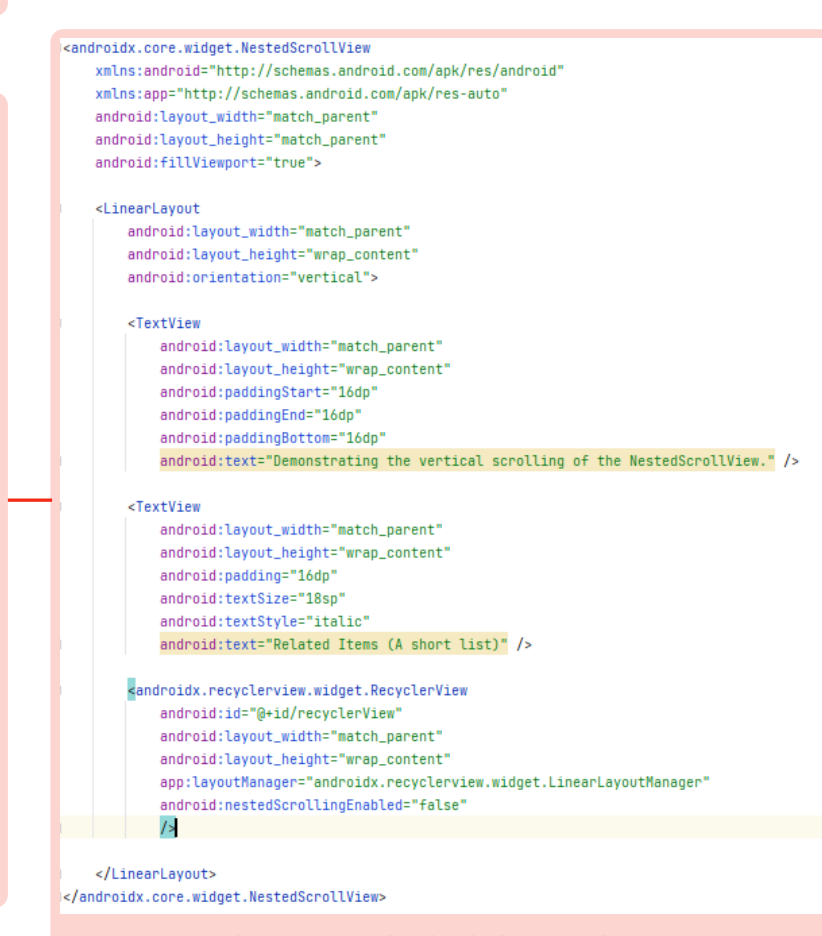
```
<ScrollView
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <!-- Your content views go here -->
        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="This is a very long text that will require

        <Button
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Click Me" />

    </LinearLayout>

</ScrollView>
```

characteristic

1. *Single child*
2. *Single direction scrolling*
   - for **Vertical** using **ScrollView** widget
   - for **Horizontal** using **HorizontalScrollView** widget
3. 

   Only use for basic scrollable content
   Ex: short lists (e.g., 5-8 items)
4. 

   Nested Scroll View

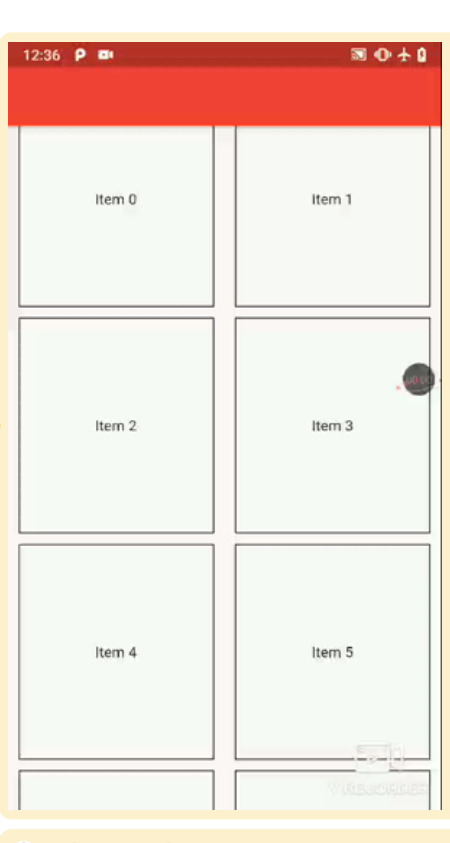


```
<androidx.core.widget.NestedScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:paddingStart="16dp"
            android:paddingEnd="16dp"
            android:paddingBottom="16dp"
            android:text="Demonstrating the vertical scrolling of the NestedScrollView." />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:textSize="18sp"
            android:textStyle="italic"
            android:text="Related Items (A short list)" />

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/recyclerView"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:layoutManager="androidx.recyclerview.widget.LinearLayoutManager"
            android:nestedScrollingEnabled="false"
            />

    </LinearLayout>
</androidx.core.widget.NestedScrollView>
```

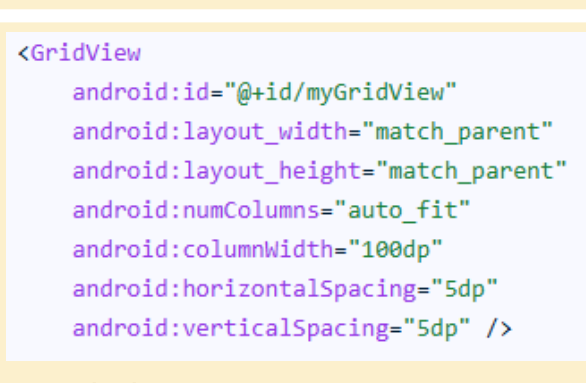
Usecase: Dynamic content (child list) inside container scrollable

② Grid View

Look



characteristic

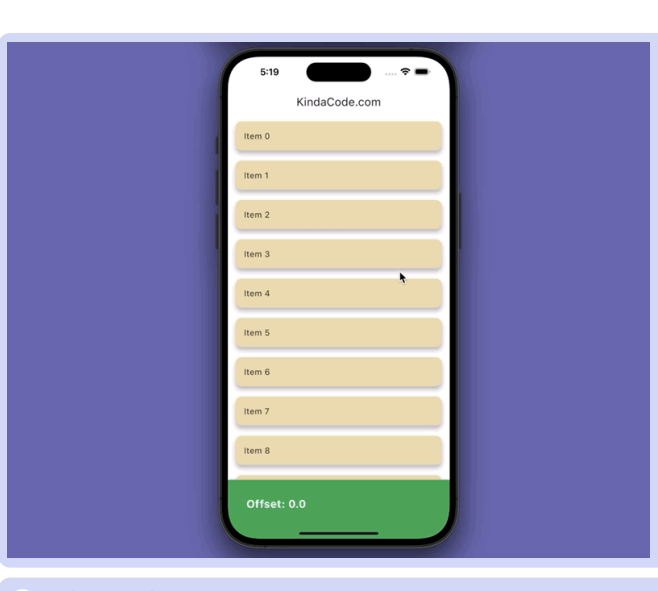
1. Display grid layout
2. 

```
<GridView
    android:id="@+id/myGridView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:numColumns="auto_fit"
    android:columnWidth="100dp"
    android:horizontalSpacing="5dp"
    android:verticalSpacing="5dp" />
```

   Customization
   - Number of Columns
   - Spacing
   - Item Selection
3. Dynamic content (adapter pattern)
4. No recycle item view (by default)
   - Can be improve by View Holder pattern
5. Poor effectively performace
6. Hard to customization

③ List View

Look

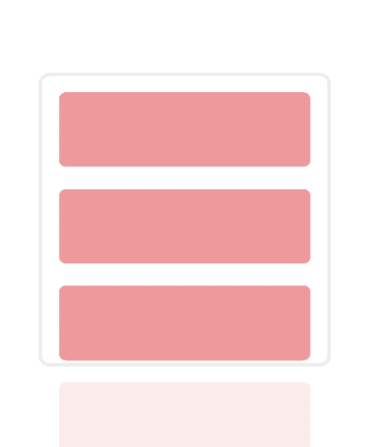


characteristic

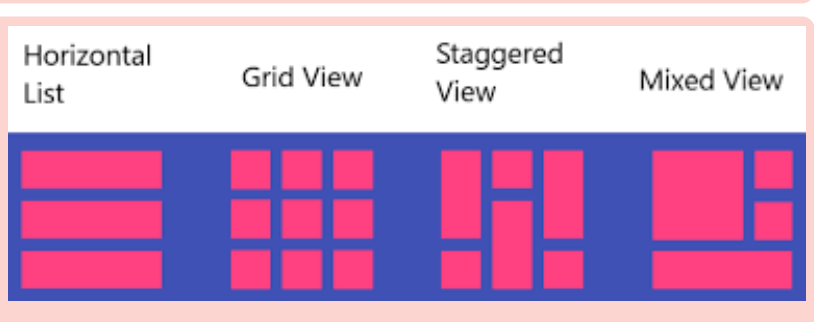
1. Display list layout
2. Dynamic content (adapter pattern)
3. No recycle item view (by default)
   - Can be improve by View Holder pattern
4. Hard to customization
5. Poor effectively performace

④ Recycler View

How it work

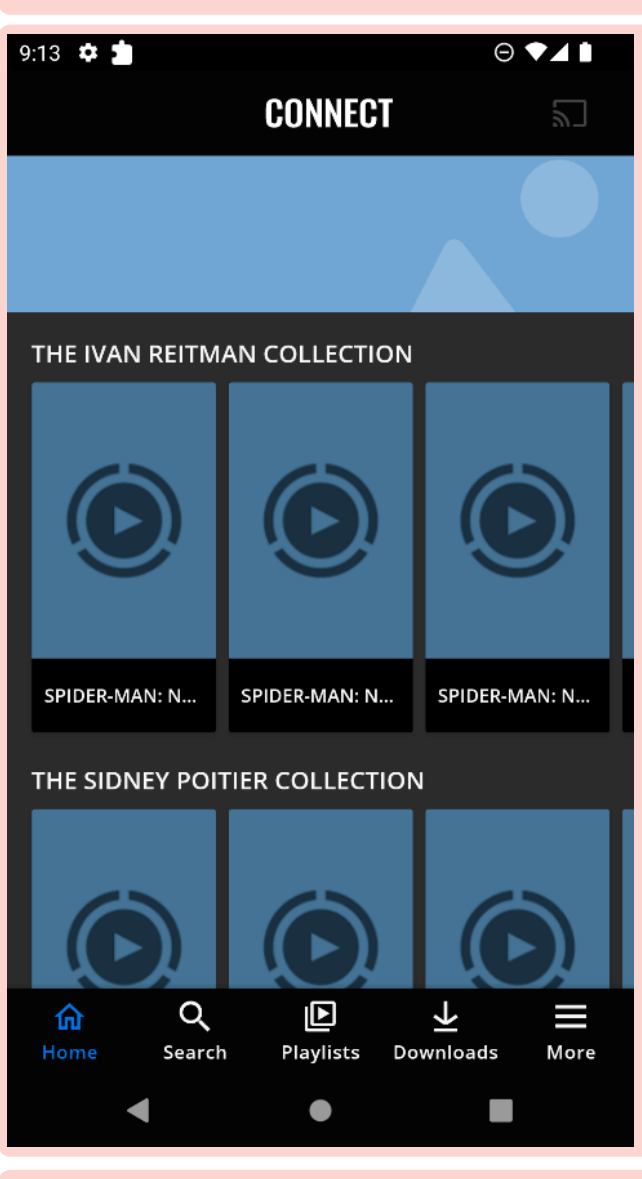


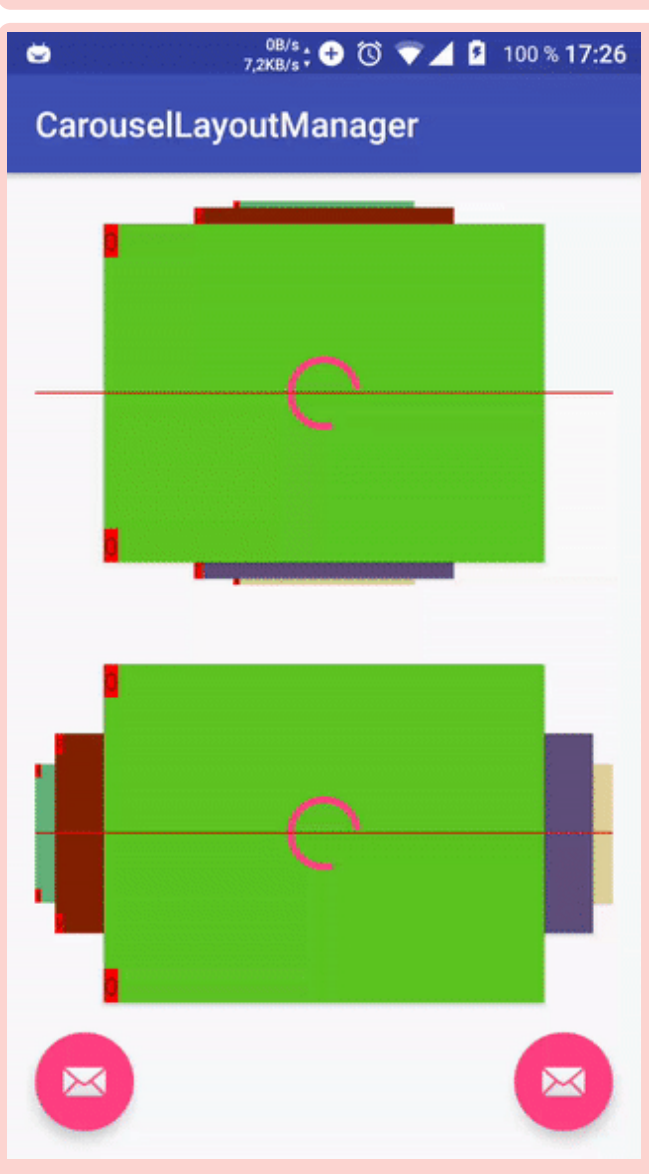
characteristic

1. Recycle view holder (by default)
2. Dynamic contents
3. Effectively performance
4. 

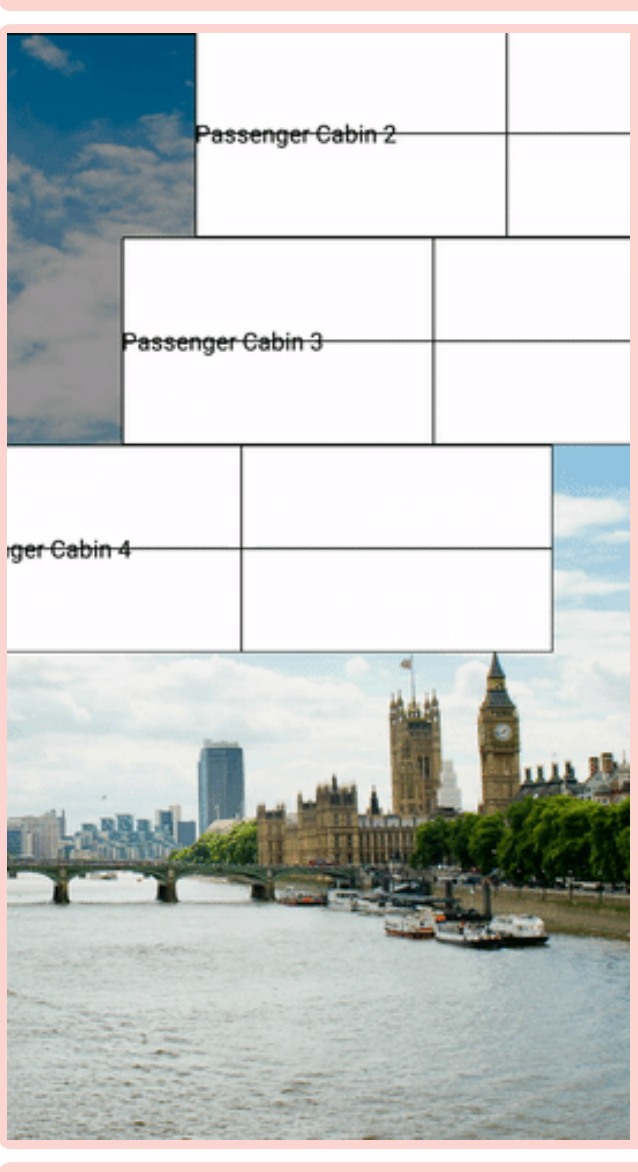
   Define how to display the children by Layout Manager
5. Can customize a lot of things
   - Header, Footer
   - Gesture handler (swipe, animation effect)
   - Layout manager (define your style)

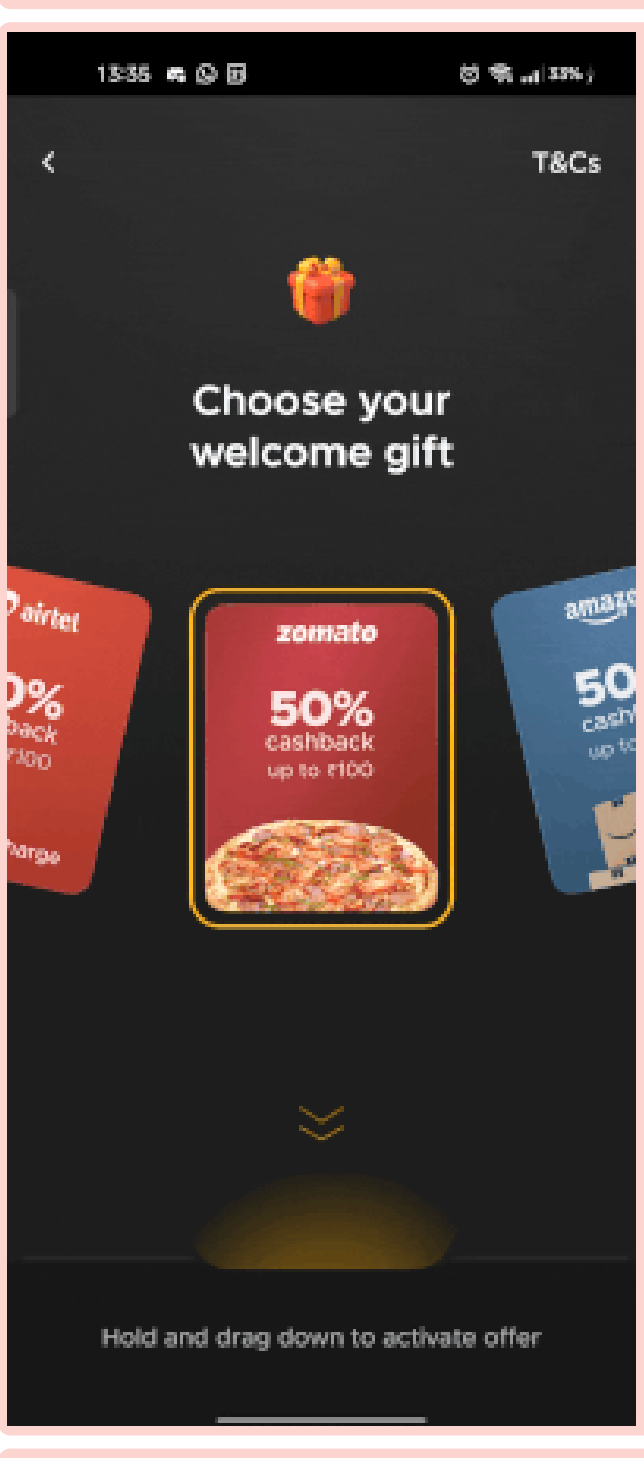
Some usecases

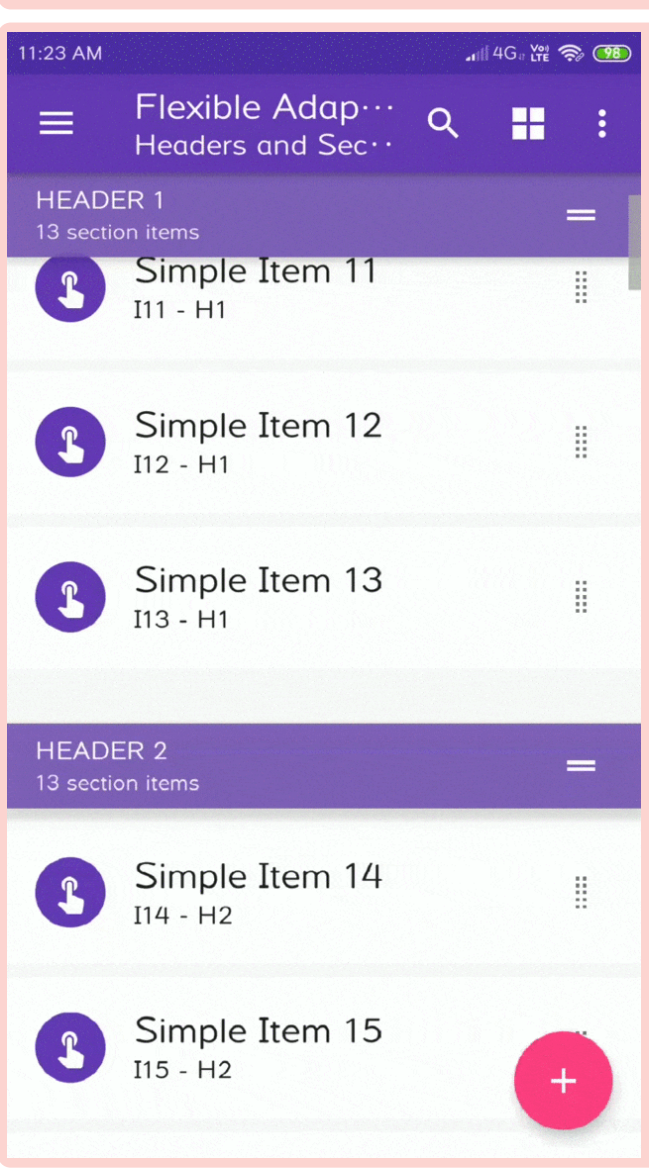




Carousel Layout Manager (Custom Layout Manager)



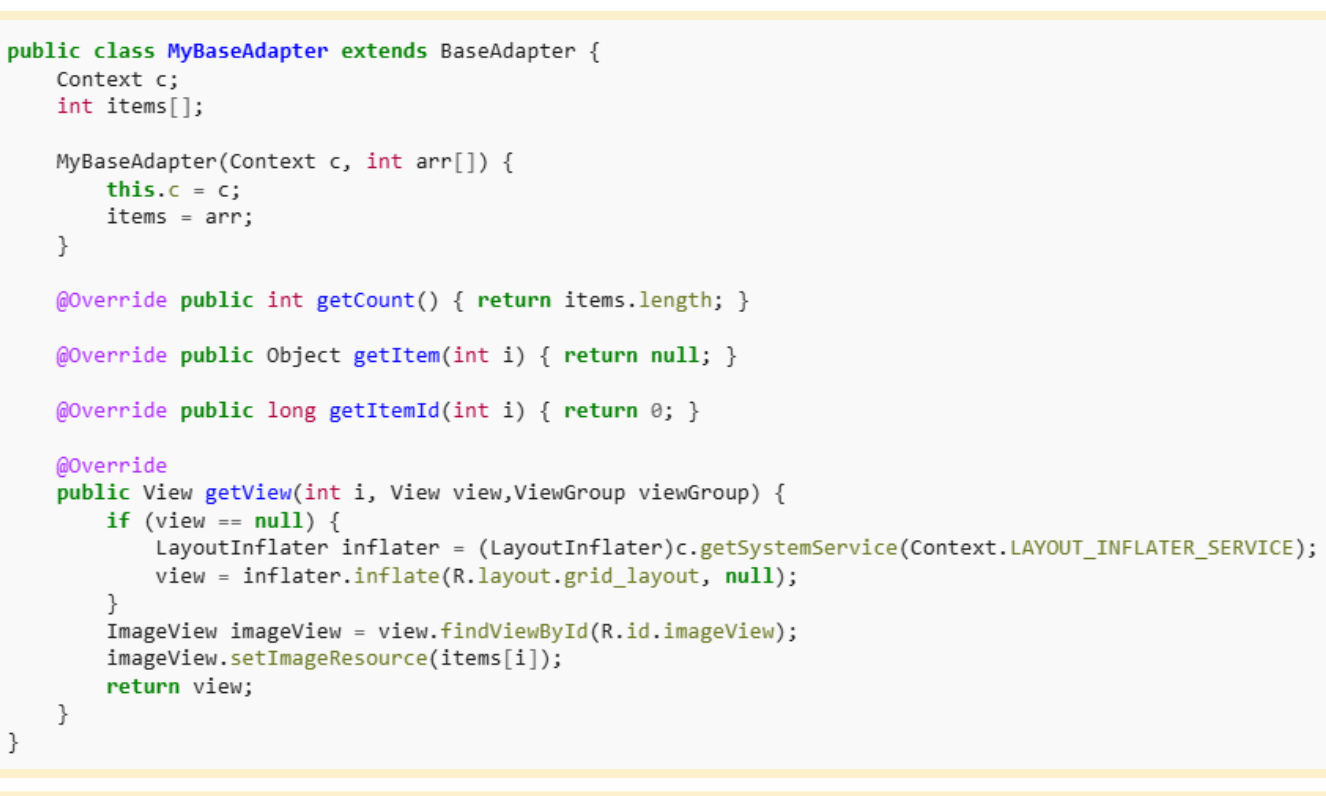




Adapter

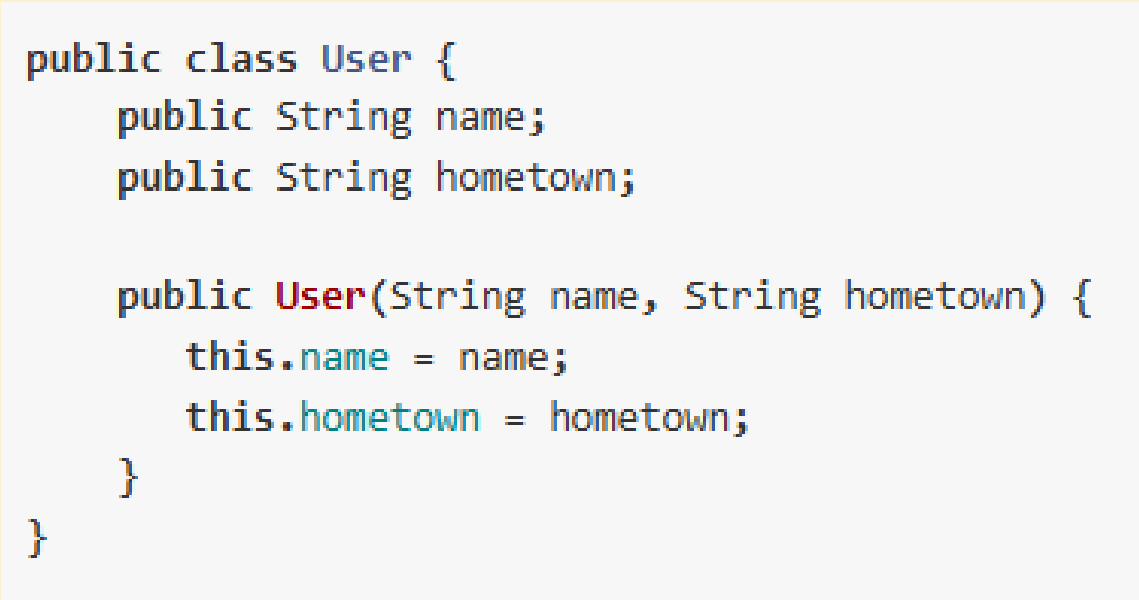
① Base Adapter

Look

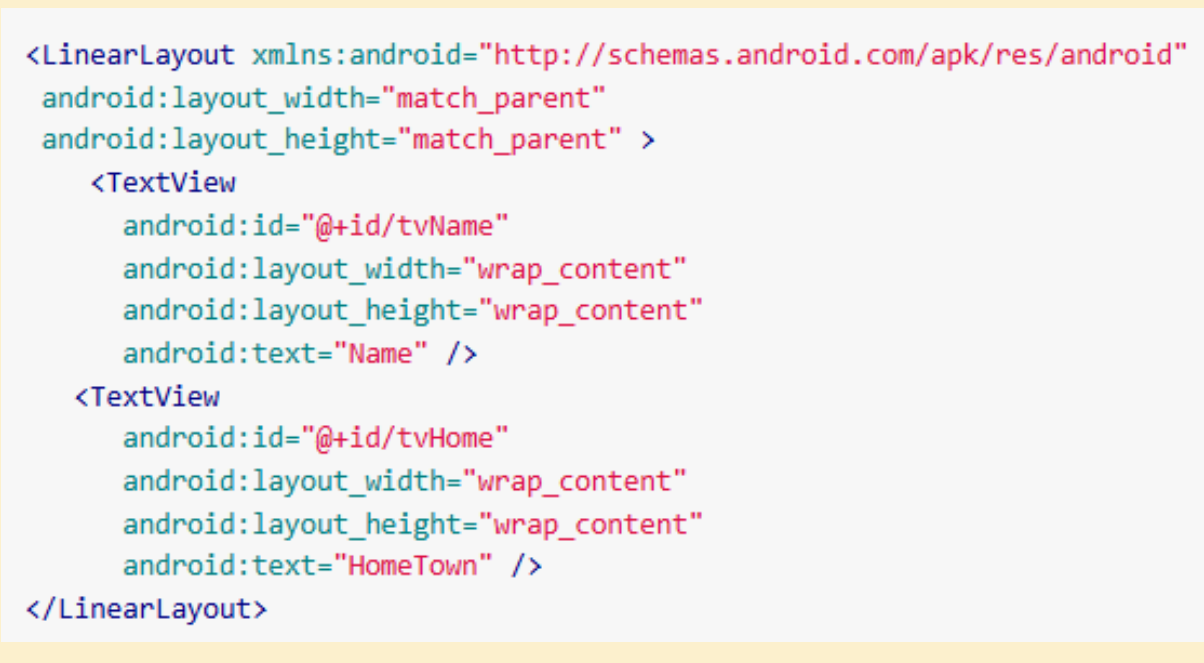


```
public class MyBaseAdapter extends BaseAdapter {
    Context c;
    int items[];

    MyBaseAdapter(Context c, int arr[]) {
        this.c = c;
        items = arr;
    }

    @Override public int getCount() { return items.length; }

    @Override public Object getItem(int i) { return null; }

    @Override public long getItemId(int i) { return 0; }

    @Override
    public View getView(int i, View view,ViewGroup viewGroup) {
        if (view == null) {
            LayoutInflater inflater = (LayoutInflater)c.getSystemService(Context.LAYOUT_INFLATER_SERVICE);
            view = inflater.inflate(R.layout.grid_layout, null);
        }
        ImageView imageView = view.findViewById(R.id.imageView);
        imageView.setImageResource(items[i]);
        return view;
    }
}
```

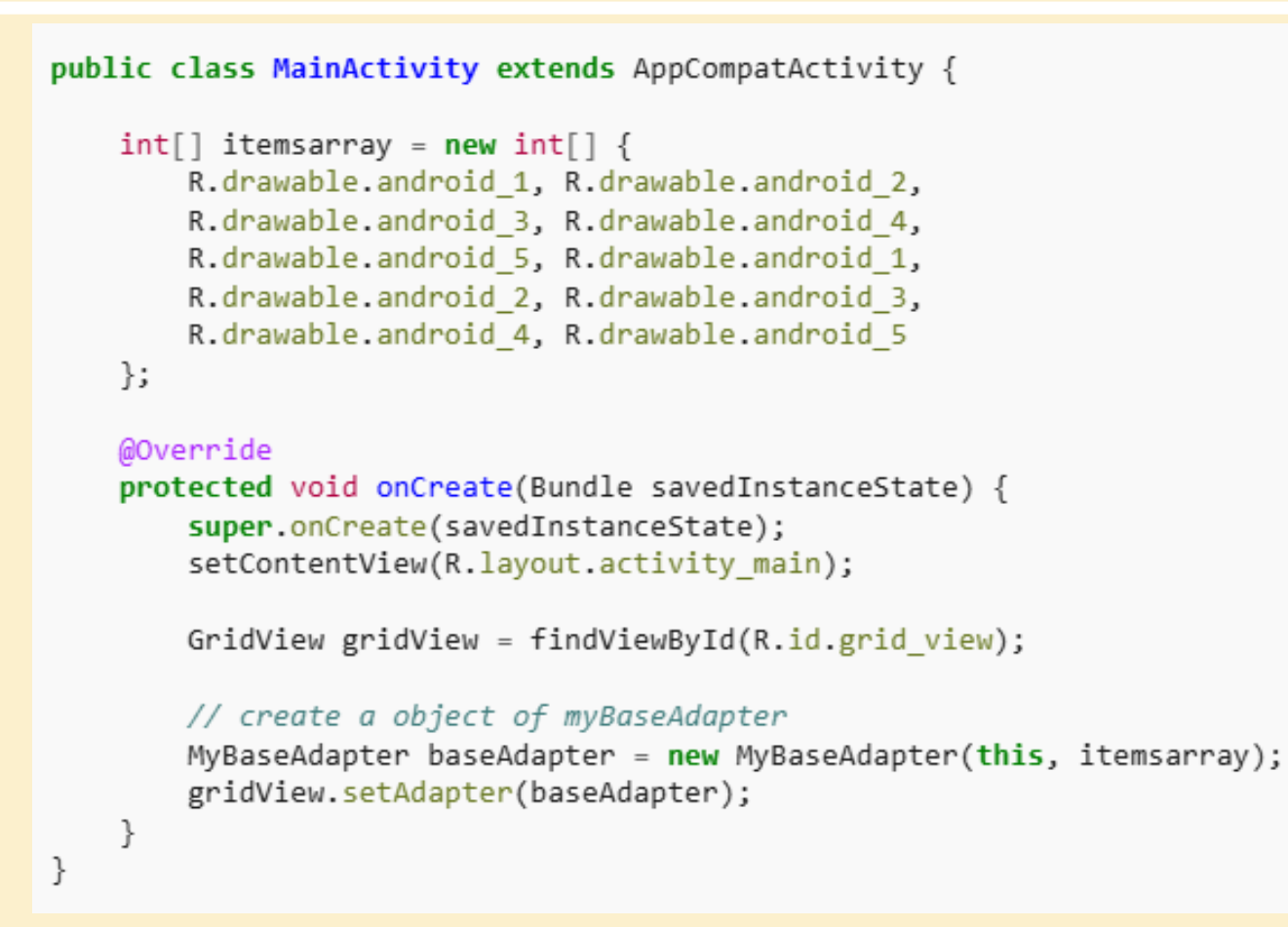
How

1. Define class base adapter ↑
2. 

```
public class User {
    public String name;
    public String hometown;

    public User(String name, String hometown) {
       this.name = name;
       this.hometown = hometown;
    }
}
```

   Define data model
3. 

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent" >
    <TextView
      android:id="@+id/tvName"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Name" />
   <TextView
      android:id="@+id/tvHome"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="HomeTown" />
</LinearLayout>
```

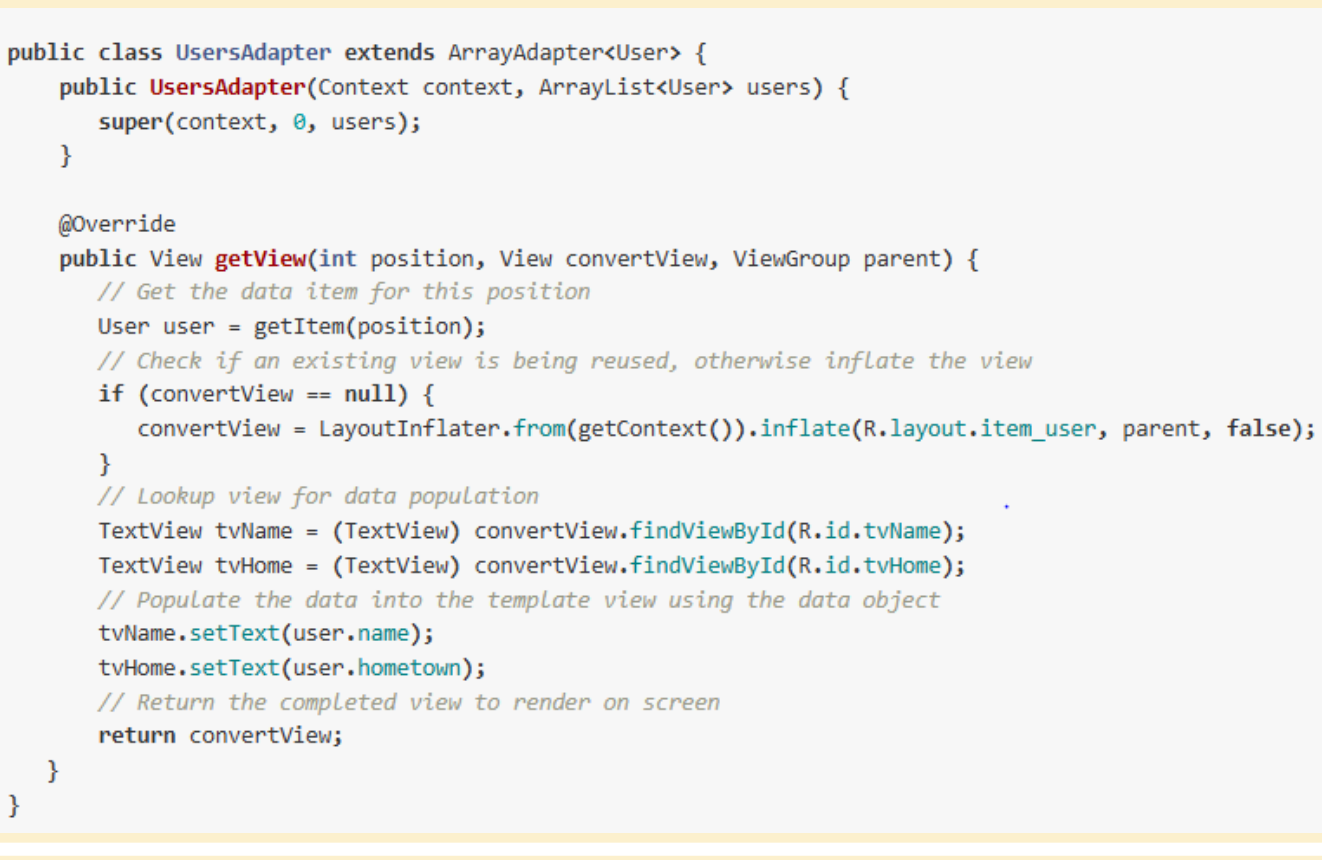
   Define layout item view
4. 

```
public class MainActivity extends AppCompatActivity {

    int[] itemsarray = new int[] {
        R.drawable.android_1, R.drawable.android_2,
        R.drawable.android_3, R.drawable.android_4,
        R.drawable.android_5, R.drawable.android_1,
        R.drawable.android_2, R.drawable.android_3,
        R.drawable.android_4, R.drawable.android_5
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        GridView gridView = findViewById(R.id.grid_view);

        // create a object of myBaseAdapter
        MyBaseAdapter baseAdapter = new MyBaseAdapter(this, itemsarray);
        gridView.setAdapter(baseAdapter);
    }
}
```

   Integrate with scrollable view

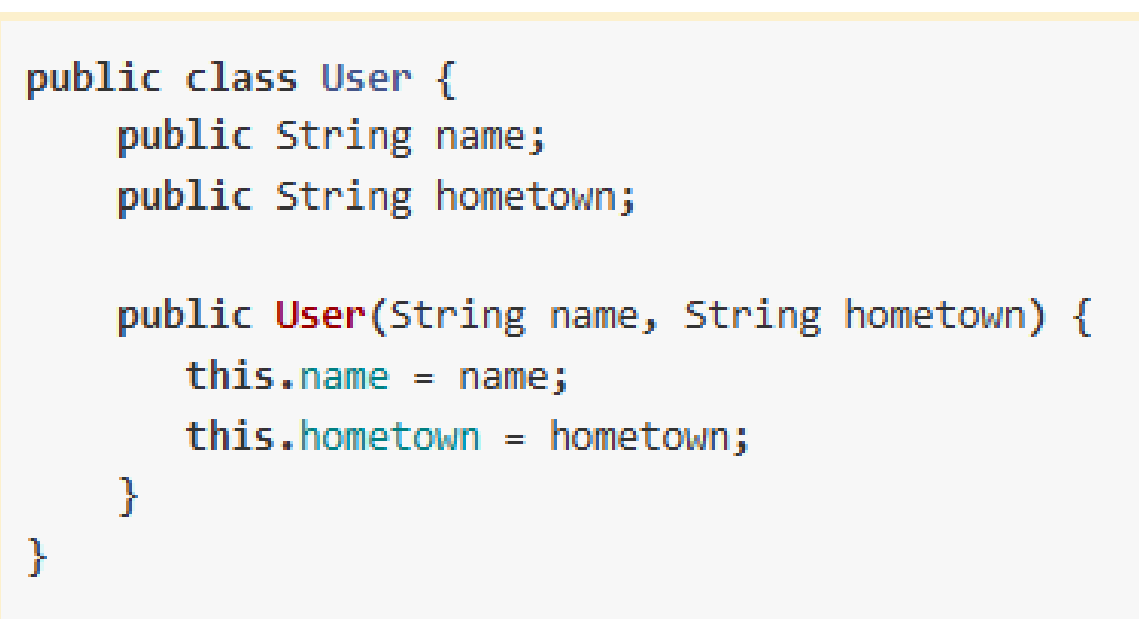
② Array Adapter

Look

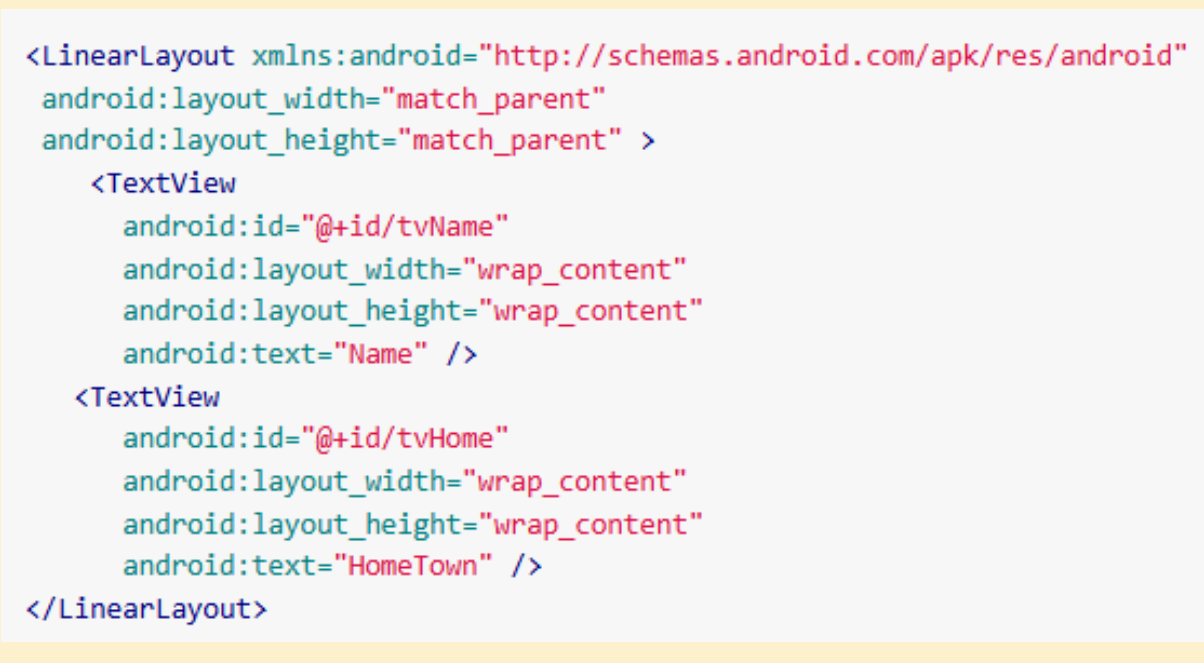


```
public class UsersAdapter extends ArrayAdapter<User> {
    public UsersAdapter(Context context, ArrayList<User> users) {
       super(context, 0, users);
    }

    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
       // Get the data item for this position
       User user = getItem(position);
       // Check if an existing view is being reused, otherwise inflate the view
       if (convertView == null) {
          convertView = LayoutInflater.from(getContext()).inflate(R.layout.item_user, parent, false);
       }
       // Lookup view for data population
       TextView tvName = (TextView) convertView.findViewById(R.id.tvName);
       TextView tvHome = (TextView) convertView.findViewById(R.id.tvHome);
       // Populate the data into the template view using the data object
       tvName.setText(user.name);
       tvHome.setText(user.hometown);
       // Return the completed view to render on screen
       return convertView;
   }
}
```

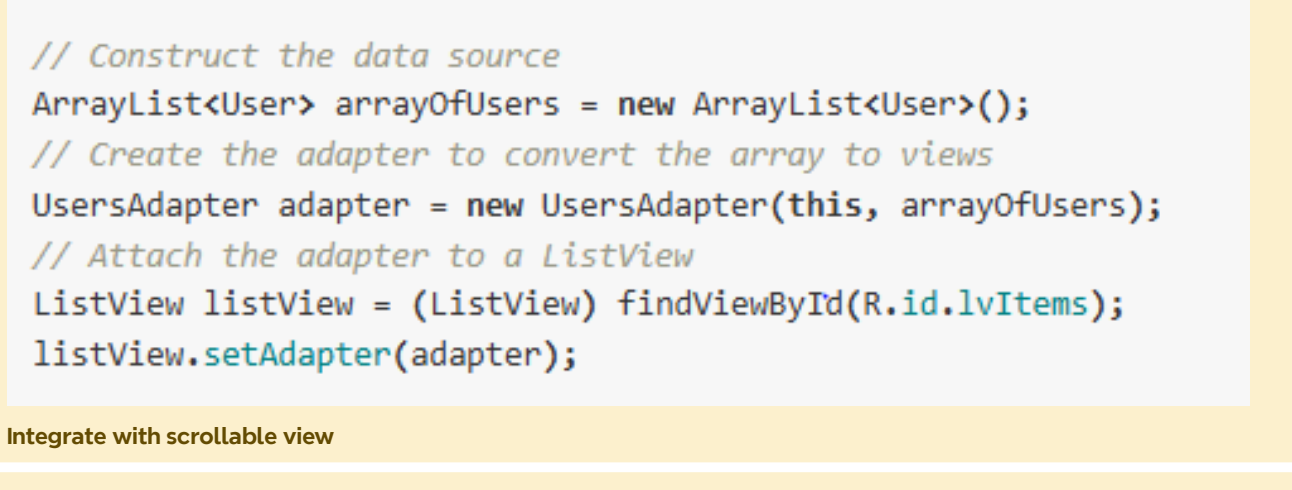
How

1. Define class array adapter ↑
2. 

```
public class User {
    public String name;
    public String hometown;

    public User(String name, String hometown) {
       this.name = name;
       this.hometown = hometown;
    }
}
```

   Define data model
3. 

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
 android:layout_width="match_parent"
 android:layout_height="match_parent" >
    <TextView
      android:id="@+id/tvName"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Name" />
   <TextView
      android:id="@+id/tvHome"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="HomeTown" />
</LinearLayout>
```

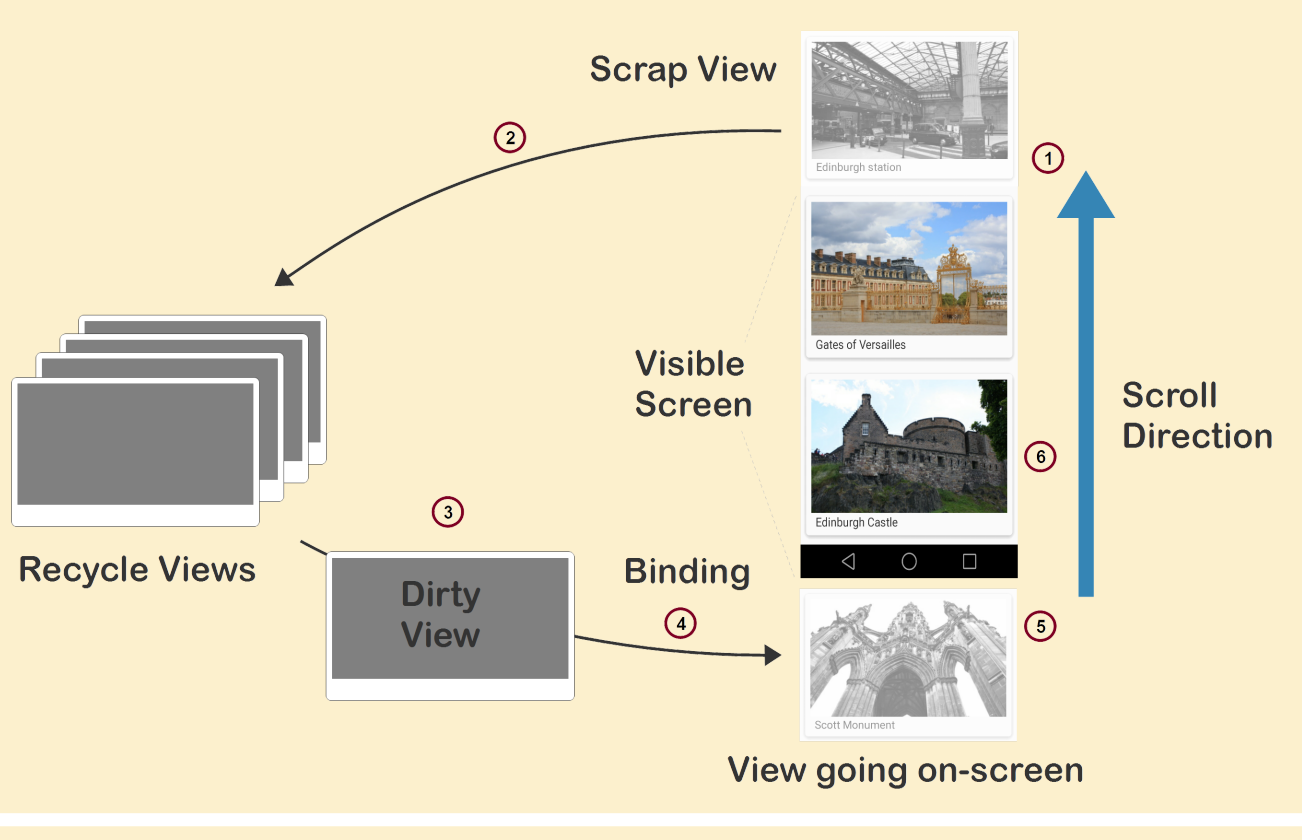
   Define layout item view
4. 

```
// Construct the data source
ArrayList<User> arrayOfUsers = new ArrayList<User>();
// Create the adapter to convert the array to views
UsersAdapter adapter = new UsersAdapter(this, arrayOfUsers);
// Attach the adapter to a ListView
ListView listView = (ListView) findViewById(R.id.lvItems);
listView.setAdapter(adapter);
```

   Integrate with scrollable view

③ RecyclerView.Adapter

How it work



How to implement

1. Create your data model
2. Create layout item view
3. Create View Holders
4. Create Recyclerview.Adapter class
   - Define onCreateViewHolder(...)
   - Define onBindViewHolder(...)
   - Define data source

Presented with xmind